

ARCHITECTURE DOCUMENT

# PPC CBE Tracking Application

System Architecture

---

**Technology stack:** Next.js, FastAPI, PostgreSQL

**Prepared by:** Athithya

**Reviewer:** Kauverysree

**Document version:** 1.0

**Status:** Draft — pending confirmation on items listed in Section 13

## Contents

---

|    |                                               |
|----|-----------------------------------------------|
| 1  | Purpose and Scope                             |
| 2  | Architecture Principles                       |
| 3  | System Context                                |
| 4  | High-Level Architecture                       |
| 5  | Component Details                             |
| 6  | Key Data Flows                                |
| 7  | Security Architecture                         |
| 8  | Deployment Architecture                       |
| 9  | Non-Functional Requirements                   |
| 10 | Technology Stack Summary                      |
| 11 | Commands Used                                 |
| 12 | Architecture Verification Checklist           |
| 13 | Open Items Requiring Stakeholder Confirmation |

# 1. Purpose and Scope

---

This document defines the end-to-end technical architecture for the PPC CBE Tracking Application, covering the polishing and plating vendor workflows: material outward to vendor, vendor processing, quality inspection, inward receipt at Titan (gate security through sub-con, SAP posting, quality assurance and storage), deviation handling, and reporting, as scoped in the project discussion notes across sixteen functional modules (M1–M16).

It is written so that another developer or architect can implement the system without needing to reverse-engineer intent. Every component, data flow, security control, and deployment decision is stated explicitly. Items still pending confirmation from stakeholders are called out in Section 13 rather than assumed.

# 2. Architecture Principles

---

| Principle              | How it is applied here                                                                                                                                                 |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Scalability</b>     | Stateless FastAPI instances behind a load balancer, horizontal scaling, read-heavy queries offloaded to cache as volume grows.                                         |
| <b>Security</b>        | JWT authentication with role-based access control, encrypted transport, encrypted secrets, input validation at every boundary, an audit trail on every state change.   |
| <b>Maintainability</b> | Domain-driven module boundaries that mirror M1–M16, typed contracts (Pydantic and TypeScript), an adapter pattern around the SAP integration.                          |
| <b>Reliability</b>     | Idempotent SAP posting operations, transactional database writes, retry and backoff on external calls, health checks, structured logging.                              |
| <b>Traceability</b>    | Every material movement from gate entry to storage is scan-driven and timestamped with an actor, matching the sign-off and audit requirements of the physical process. |

### 3. System Context

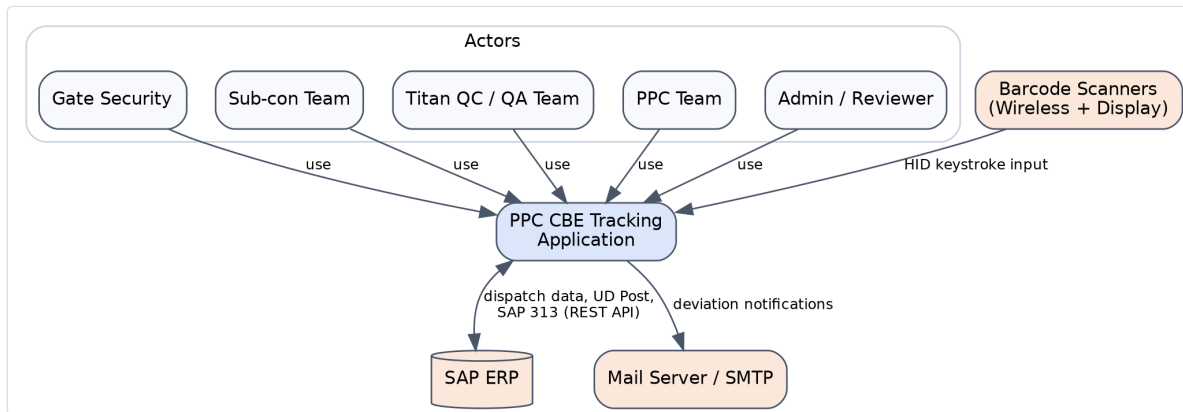


Figure 1 — System context: actors, the application, and external systems.

SAP ERP is the source of dispatch data and the destination for UD Post and SAP 313 status updates. The integration is confirmed as REST API; exact endpoints, authentication mechanism, and payload schema are pending SAP team confirmation (see Section 13). The mail server is used for outbound vendor deviation notifications only. Both barcode scanner models specified for this project — the wireless scanner and the scanner with display — operate in HID (keyboard-wedge) mode, feeding scanned codes directly into the active input field.

## 4. High-Level Architecture

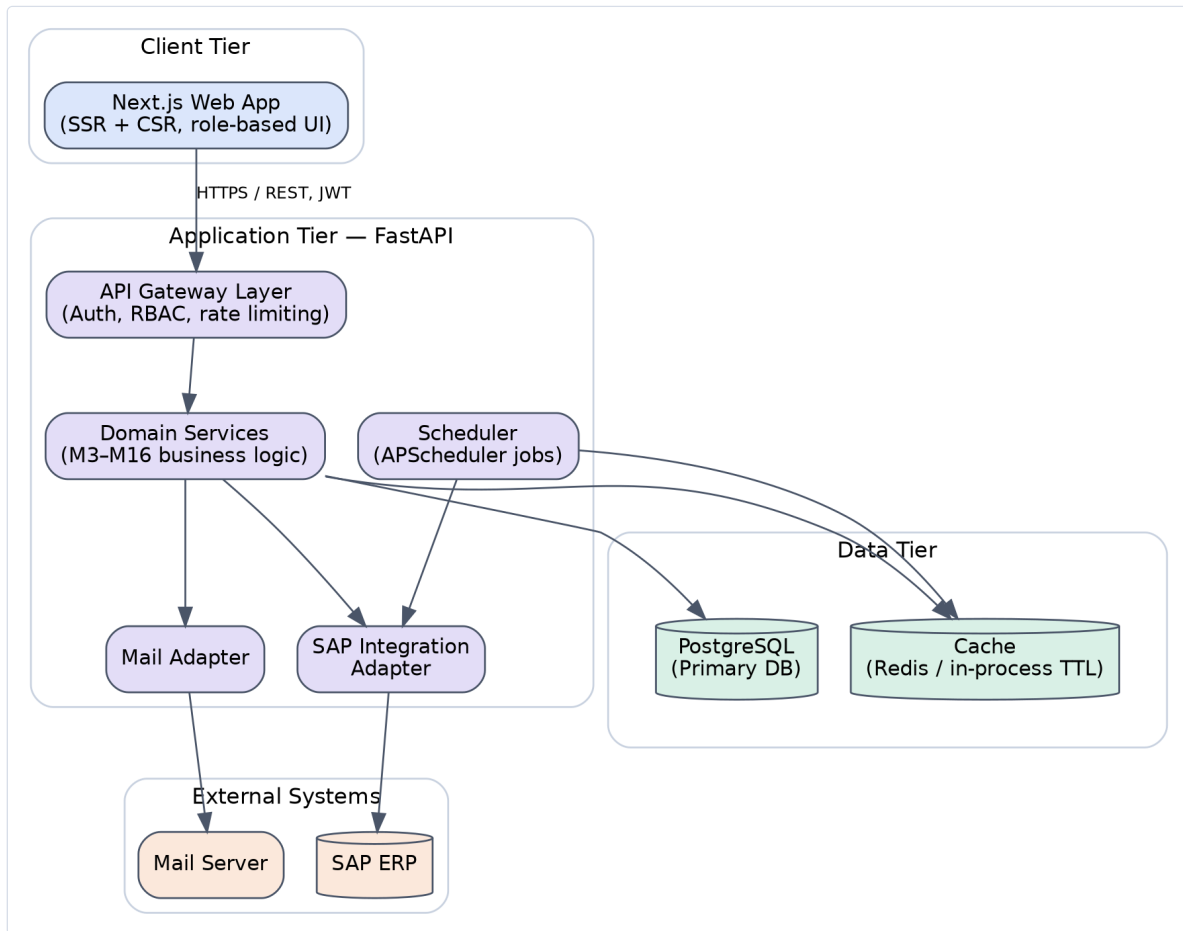


Figure 2 — Layered architecture: client, application, data, and external tiers.

| Layer                | Technology                   | Responsibility                                                                   |
|----------------------|------------------------------|----------------------------------------------------------------------------------|
| Client               | Next.js (App Router)         | UI rendering, scan input capture, client-side validation, role-based navigation. |
| API Gateway          | FastAPI dependency layer     | JWT verification, RBAC enforcement, request validation, rate limiting.           |
| Domain Services      | FastAPI routers and services | Business rules per module (M3–M16).                                              |
| Integration Adapters | httpx-based clients          | Isolate SAP and mail-server volatility from core business logic.                 |
| Scheduler            | APScheduler (in-process)     | Periodic SAP sync, KPI cache refresh.                                            |
| Data                 | PostgreSQL                   | System of record, audit trail.                                                   |
| Cache                | Redis or a TTL cache         | Master data and KPI caching.                                                     |

## 5. Component Details

### 5.1 Frontend — Next.js

Routing is organized by module rather than by generic CRUD pattern, since the module list maps closely to distinct operational workflows:

```
app/
  (auth)/login/
  (dashboard)/
    dashboard/           # M2
    sap-outward/         # M3
    receiving/           # M4
    dc-verification/     # M5
    physical-verification/ # M6
    deviations/          # M7
    qc-acknowledgement/  # M8
    sap-processing/       # M9
    ppc-collection/      # M10
    qa-inspection/       # M11
    zqmtl1/              # M12
    storage/             # M13
    reports/             # M14
    masters/             # M15
    settings/            # M16
  (floor)/               # kiosk-style routes for scan-first stations
    gate-entry/
    subcon-scan/
```

Server components are used for read-heavy screens such as the dashboard, reports, and masters. Client components are used for scan-driven, multi-step forms such as receiving, deviation handling, and verification.

Both scanner models operate in HID mode, emitting keystrokes into whichever input is focused, terminated by Enter. A shared scanning hook listens for rapid keystroke bursts followed by a terminator on a focused input, distinguishes this from manual typing, and forwards the decoded code to the active form. No camera access or vendor SDK is required.

Authentication uses a JWT access token stored in an httpOnly, secure, same-site cookie rather than local storage, which limits exposure to token theft through cross-site scripting. A refresh token is rotated on expiry. Route middleware guards all dashboard and floor routes.

TanStack Query handles server state, with per-data-type cache duration (see Section 6). Zustand is used only for in-progress, multi-step form state such as a deviation flow spanning several screens.

For performance, heavy or rarely used bundles (reports, masters) are loaded with dynamic imports, images use Next.js image optimization, and long lists are virtualized rather than rendered in full.

Gate security and sub-con scanning stations are recommended to use a separate, lightweight route group with large touch targets and a scan-first flow, distinct from the fuller navigation shell used by office and administrative users.

## 5.2 Backend — FastAPI

```

app/
  core/
    config.py          # env-driven settings (Pydantic Settings)
    security.py        # JWT issue/verify, password hashing, RBAC deps
    database.py        # async SQLAlchemy engine/session
    logging.py         # structured logging config
  modules/
    auth/              dashboard/      sap_outward/
    receiving/         dc_verification/ physical_verification/
    deviations/        qc_acknowledgement/ sap_processing/
    ppc_collection/    qa_inspection/    zqmtl1/
    storage/           reports/          masters/          settings/
    router.py          schemas.py      models.py         service.py        repository.py
  integrations/
    sap/
      client.py         # real REST client
      mock_client.py    # dev/test double
      interface.py      # abstract base; service layer depends on this
    mail/
      client.py
  workers/
    scheduler.py       # APScheduler job registration
    jobs.py            # sap_sync_job, kpi_refresh_job, etc.
  middleware/
    error_handlers.py
    rate_limit.py
  main.py              # app factory, router + middleware registration

```

Services depend on an abstract SAP client interface rather than the concrete REST client. This is the single most important decision given that the SAP field-level scope is still pending confirmation: it allows the rest of the system to be built, tested, and demonstrated against a mock client today, with no refactor required once the real SAP endpoints are confirmed.

Authentication follows the OAuth2 password flow, issuing a short-lived access token and a longer-lived refresh token. Route dependencies enforce both authentication and role checks — for example, only the QC role can complete the M8 acknowledgement step, and only gate security can log a gate entry.

Routes, the database layer, and the SAP client all use async I/O, since SAP calls and mail sends are the slowest points in the request path.

No task queue is used. FastAPI's background tasks handle fire-and-forget work such as deviation emails. APScheduler, running in-process, handles the one genuinely time-based job — periodic SAP dispatch-data sync — which background tasks alone cannot cover, since they only run in response to a request.

Several business rules are enforced in the service layer rather than left to the frontend: the delivery challan line-item cap of thirteen, which triggers an automatic split; the excess/less quantity comparison, which branches into the deviation workflow; and scan-sequence enforcement, so that, for example, a bin transfer scan cannot be recorded before the corresponding physical verification scan exists for that white box.

API contracts use Pydantic schemas kept separate from the ORM models. This matters particularly for the SAP-sourced fields in M3, most of which remain unconfirmed — the request and response contract can be defined and versioned independently of whatever SAP eventually returns. SAP posting endpoints accept an idempotency key so that a retried request after a timeout does not result in a duplicate post.

### 5.3 Database — PostgreSQL

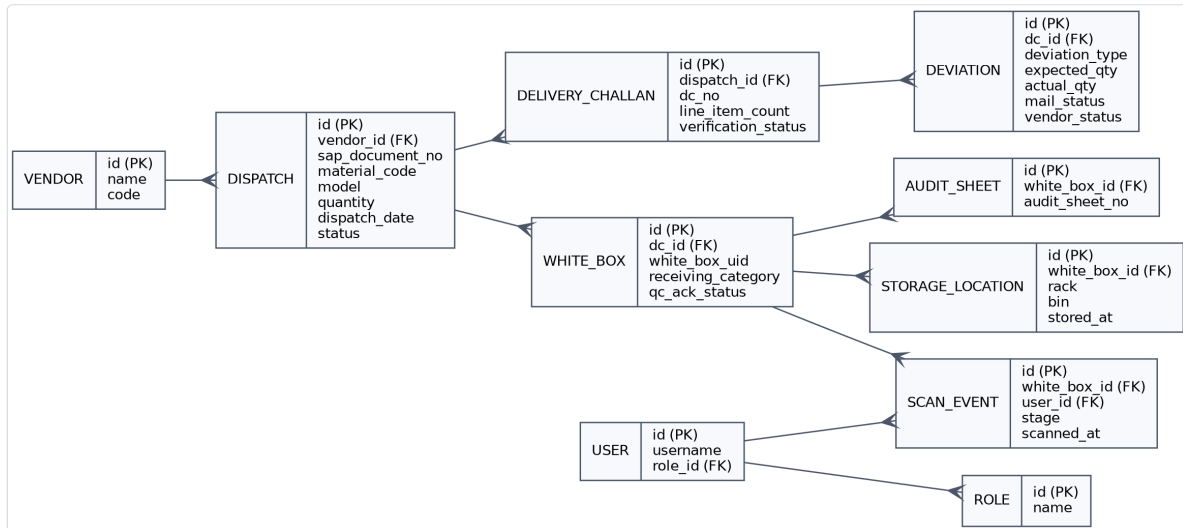


Figure 3 — Core entity relationships (illustrative; to be finalized against confirmed SAP fields).

Every transactional table carries created-by, created-at, updated-by, and updated-at columns, given how sign-off and traceability-heavy the underlying process is. A separate scan-event table, rather than overloaded status columns, captures every scan as an immutable event. This provides a full audit trail per white box across gate entry, physical verification, bin transfer, storage, and issue, and lets reporting be built from event history rather than current state alone.

Indexes are required on white box UID, DC number, SAP document number, vendor, and all status or enumerated columns used in filters, since these are the fields hit on every scan and every list or search screen. Schema changes are managed through versioned Alembic migrations; manual schema edits in production are not permitted. Materialized views for dashboard KPI counts should be introduced once transaction volume grows, refreshed by the scheduler rather than computed live on every request.

### 5.4 Caching Layer

| Data                                             | Cache location          | TTL / invalidation                                                     |
|--------------------------------------------------|-------------------------|------------------------------------------------------------------------|
| Master data (vendor, material, model, rack, bin) | Redis or in-process TTL | Thirty minutes or more; invalidate on write.                           |
| Dashboard KPI counts                             | Redis                   | Thirty to sixty seconds, refreshed in the background by the scheduler. |
| SAP dispatch data snapshot                       | Redis                   | Invalidated on the next successful sync rather than purely by time.    |
| Session and token revocation list                | Redis                   | TTL equal to token expiry.                                             |

The choice between Redis and an in-process cache depends on the expected deployment topology (single instance versus multiple FastAPI instances), which is one of the items pending confirmation in Section 13.

## 6. Key Data Flows

### 6.1 Material Inward — Happy Path

The flow below traces a White Box from gate entry through to a pending SAP 313 status, in the order the steps occur.

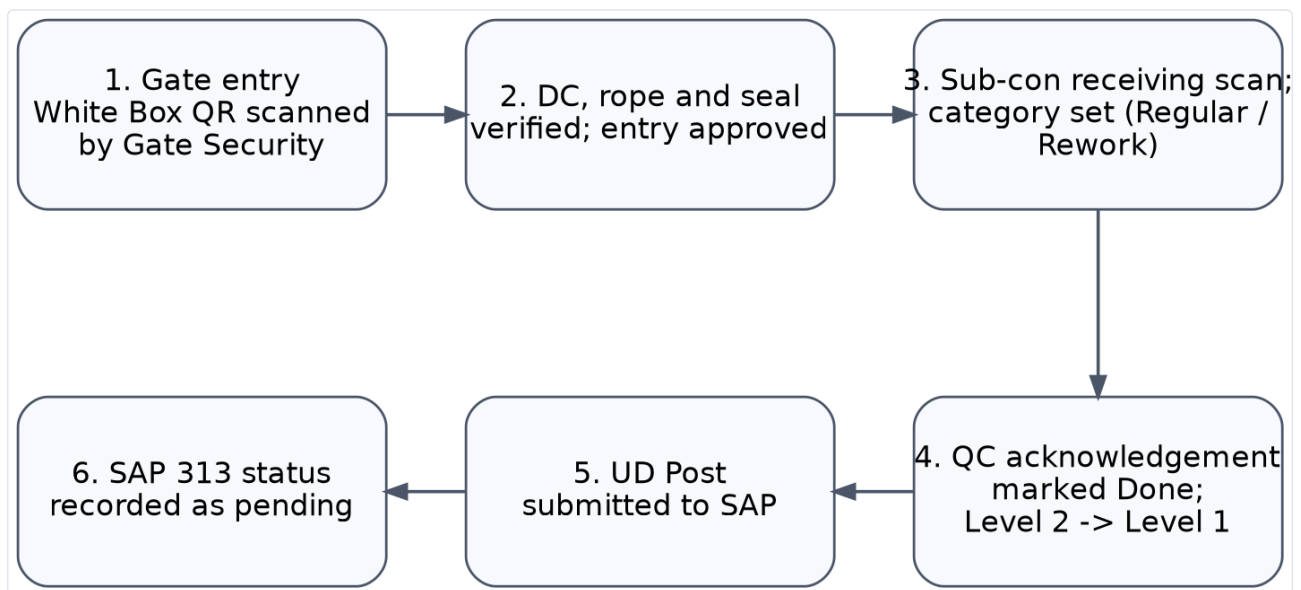


Figure 4 — Material inward flow, gate entry through SAP 313 posting (read left to right, then left to right on the second row).

## 6.2 Deviation Flow — Quantity Shortfall

The flow below shows the branch that is triggered when the received quantity is less than the expected quantity during sub-con verification.

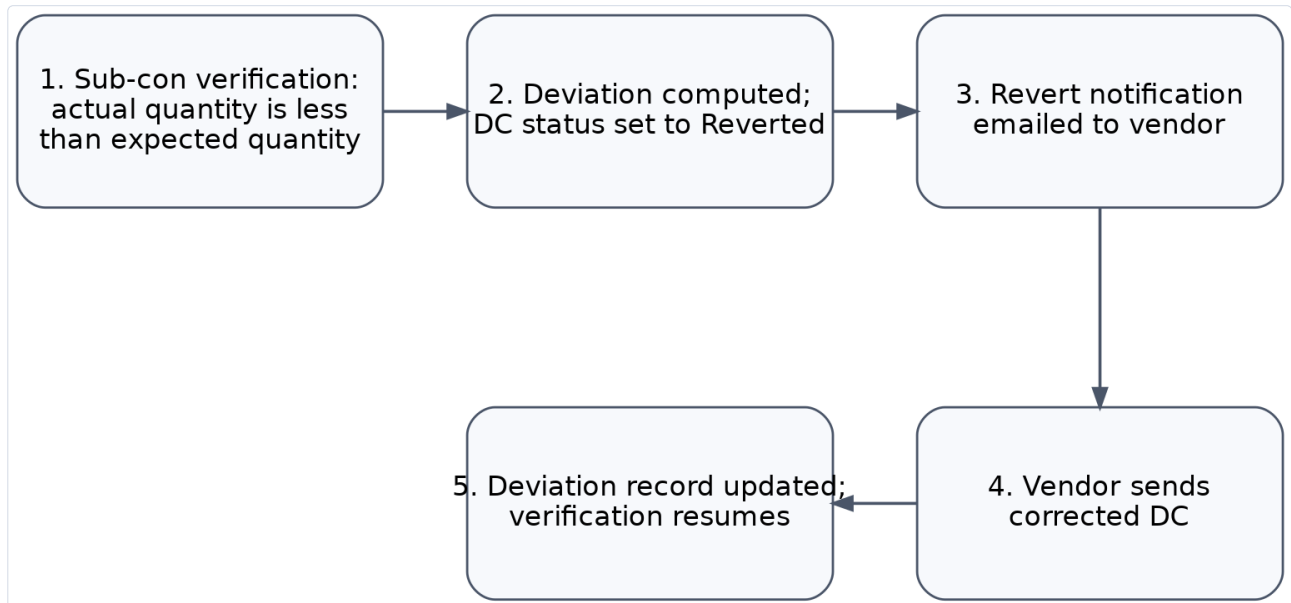


Figure 5 — Deviation handling, received quantity less than expected (read left to right, then left to right on the second row).

## 7. Security Architecture

### 7.1 Authentication and Authorization

The OAuth2 password grant issues a short-lived JWT access token, together with a refresh token that rotates on use and is stored in an httpOnly, secure cookie. Role-based access control is mapped to the operational roles in the process — gate security, sub-con, QC, QA, PPC, and admin — and is enforced server-side on every route. Hiding a button on the frontend is a usability measure, not a security control. Passwords are stored using bcrypt or argon2 hashing; plaintext or reversible encryption is not used.

### 7.2 Transport and Data Protection

TLS is enforced across all connections — client to API, and API to SAP and mail — with no plaintext HTTP beyond local development. Secrets, including database credentials, the SAP API key, the JWT signing key, and SMTP credentials, are stored as environment variables injected through a secrets manager and are never committed to source control. Database encryption at rest depends on the hosting environment and is confirmed alongside the infrastructure decision.

### 7.3 API Security

Every endpoint validates input through Pydantic schemas, rejecting malformed or oversized payloads before they reach business logic. Login endpoints are rate limited to reduce brute-force risk. CORS is restricted to known frontend origins, with no wildcard in production. SQL injection is mitigated structurally through the ORM's parameterized queries; raw string-interpolated SQL is not used.

### 7.4 Device and Scan Input Security

Because both scanners operate in HID mode, any keyboard input can technically reach a scan-input field. Scanned codes are validated server-side against the expected format and checksum before being acted on, rather than trusting that all HID input represents a genuine scan.

### 7.5 Audit and Traceability

Every state-changing action — QC acknowledgement, deviation revert, storage assignment — is written to an append-only audit table with the actor, timestamp, and before/after status. This mirrors the sign-off-heavy nature of the physical process and supports compliance and dispute review. Structured, centralized logging covers all API requests, SAP integration calls, and background job runs, which is necessary for diagnosing SAP-related failures without guesswork.

### 7.6 Environment Isolation

Configuration and credentials are kept separate per environment. The mock SAP client is used in development and staging until the real SAP integration is confirmed and reachable; non-production environments are not pointed at the live SAP instance without explicit sign-off.

## 8. Deployment Architecture

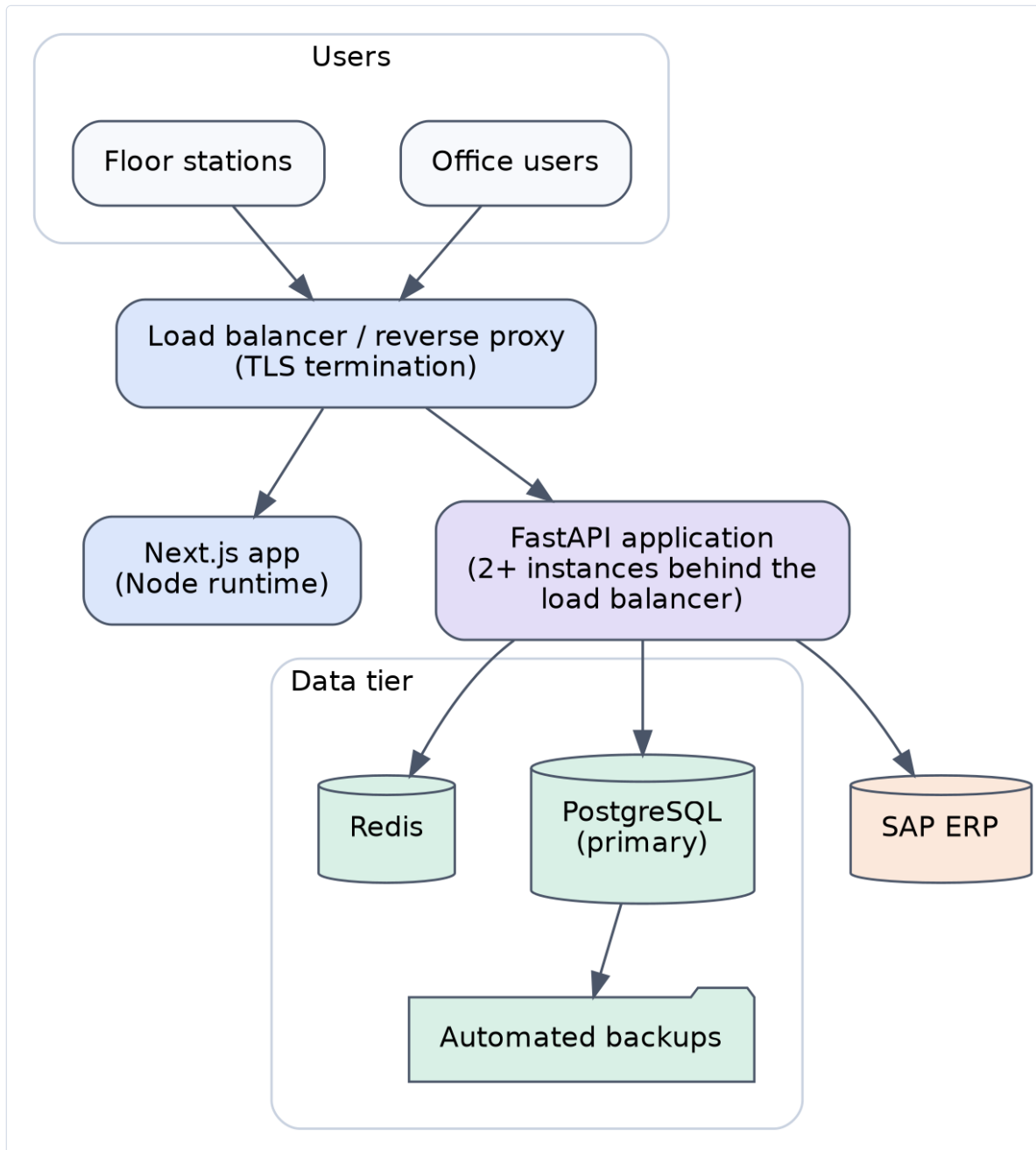


Figure 6 — Deployment topology across client, application, and data tiers.

FastAPI runs under multiple Uvicorn workers (managed by Gunicorn, or Uvicorn's own multi-worker mode) for process-level concurrency. A load balancer or reverse proxy terminates TLS and routes requests to the Next.js application and the FastAPI service separately. PostgreSQL runs as a single primary at launch, with a read replica introduced only if and when reporting query load justifies it. The target hosting environment — on-premises versus cloud — is one of the items pending confirmation in Section 13; the architecture described here is portable across either choice.

## 9. Non-Functional Requirements

| Attribute                 | Target (proposed — to be confirmed with stakeholders)                      |
|---------------------------|----------------------------------------------------------------------------|
| Availability              | 99% during working hours, reflecting factory floor dependency.             |
| API response time         | Under 300 ms for scan-driven endpoints, 95th percentile.                   |
| Concurrent floor stations | Ten to twenty concurrent scan stations at initial launch.                  |
| Data retention            | Full audit trail retained indefinitely, pending a formal retention policy. |
| Backup frequency          | Daily automated PostgreSQL backups, with restore tested quarterly.         |

## 10. Technology Stack Summary

| Layer                   | Choice                                      | Notes                                         |
|-------------------------|---------------------------------------------|-----------------------------------------------|
| Frontend                | Next.js (App Router), TypeScript            | Server-side rendering with client components. |
| State and data fetching | TanStack Query, Zustand                     | Server cache plus local multi-step state.     |
| Backend                 | FastAPI (async), Python                     | REST API.                                     |
| ORM and migrations      | SQLAlchemy 2.0 (async), Alembic             |                                               |
| Database                | PostgreSQL                                  |                                               |
| Cache                   | Redis, or an in-process TTL cache initially |                                               |
| Scheduling              | APScheduler, in-process                     | No task queue introduced at this stage.       |
| Authentication          | OAuth2 with JWT                             | httpOnly cookies.                             |
| Scanning                | HID / keyboard-wedge scanners               | Wireless and display models.                  |
| Integration             | httpx (async REST client) for SAP           | Adapter pattern.                              |

## 11. Commands Used

---

### Project scaffolding

```
# Next.js frontend
npx create-next-app@latest ppc-cbe-frontend --typescript --app --tailwind

# FastAPI backend
python -m venv venv && source venv/bin/activate
pip install fastapi uvicorn[standard] sqlalchemy asyncpg alembic \
    pydantic-settings httpx apscheduler python-jose[cryptography] passlib[bcrypt]

# Alembic init
alembic init migrations
```

### Database

```
# Create database
createdb ppc_cbe

# Generate a migration after model changes
alembic revision --autogenerate -m "add white_box and scan_event tables"
alembic upgrade head
```

### Running services — development

```
# Backend
uvicorn app.main:app --reload --port 8000

# Frontend
npm run dev
```

### Running services — production

```
# Backend, Gunicorn managing Uvicorn workers
gunicorn app.main:app -k uvicorn.workers.UvicornWorker -w 4 -b 0.0.0.0:8000

# Frontend
npm run build && npm run start
```

### Testing and verification

```
pytest -v                # backend test suite
npm run test              # frontend test suite
npm run lint && npm run type-check
```

## 12. Architecture Verification Checklist

---

To be completed before the architecture is signed off for build.

### Functional coverage

- Every module M1–M16 maps to a named frontend route and a backend service, consistent with Sections 5.1 and 5.2.
- The DC line-item split rule, the excess/less deviation logic, and scan-sequence enforcement are implemented server-side, not only in the UI.

### Security

- All routes require authentication except login and health checks.
- Role checks are verified server-side for every state-changing action, not only through hidden UI controls.
- Secrets are confirmed absent from source control.
- TLS is enforced in all non-local environments.

### Reliability

- SAP calls have a configured timeout and retry/backoff policy.
- SAP posting endpoints are confirmed idempotent through a duplicate-request test.
- Database migrations are reversible and tested against a staging copy before production.

### Performance

- Indexes exist on all scan and search fields — white box UID, DC number, SAP document number, and vendor/status columns.
- List endpoints such as reports and DC verification are checked for N+1 query patterns.
- Master data and KPI caching are in place with defined invalidation triggers.

### Operational readiness

- Health check endpoints exist for load balancer probes.
- Structured logs capture request ID, actor, and outcome for every SAP call.
- The backup and restore procedure has been tested at least once before go-live.

## 13. Open Items Requiring Stakeholder Confirmation

---

Carried forward from the project discussion notes and this design process:

1. **SAP field-level scope** — most M3 fields (DC, PO, model, version, quantity, vendor name, timestamp) remain marked as unconfirmed in the project discussion notes. REST has been confirmed as the integration style, but exact endpoints, authentication, and payload structures are not yet available.

2. **Plating scope contradiction** — the project discussion notes mark plating as unconfirmed by the Titan team, while the Phase 1 requirement table describes it in full detail across five stages. A single source of truth is needed.
3. **Module numbering conflict** — the M1–M16 module table and the separate action/functional table number M4 and M5 differently. This needs reconciliation before schema and routing are finalized.
4. **File and document storage strategy** — audit sheets, DC PDFs, and signed SAP 313 printouts need a storage decision (object storage, filesystem, or database blob) and clarity on where these documents are generated or uploaded from.
5. **Hosting environment** — on-premises versus cloud deployment, which affects the deployment specifics in Section 8 but not the core architecture.
6. **Return process** — the return process referenced in the project discussion notes (stage 53) remains unconfirmed, with no description available yet.